

Model Predictive Control Report

Group AM:
Masa Duric 407942
Aayushi Kedar Barve 415185
Andrea Tarabay 391738

Contents

1	Introduction	2
2	Deliverable 2.1: Separation of the system into independent subsystems	2
3	Deliverable 3	3
3.1	MPC Velocity Controllers	3
3.2	Tracking	5
3.3	PID contoler	6
4	Deliverable 4.1: Simulation with Nonlinear Rocket	7
4.1	Design and tuning	7
4.2	Plots	8
5	Deliverable 5: Offset-free tracking	9
5.1	Offset-free tracking in z via constant disturbance	9
5.2	Deliverable 5.2: Offset-free tracking with changing mass of the rocket	12
6	Deliverable 6: Robust MPC for Landing	13
6.1	Robust tube MPC for subsystem Z	13
6.2	Deliverable 6.2: Robust Tube MPC for landing with nominal position controllers for X, Y, roll	15
7	Deliverable 7: Nonlinear MPC	16
7.1	Design Procedure	16
7.2	Choice of Tuning Parameters	17
7.3	Performance Analysis	18
7.4	NMPC vs Robust MPC	19

1 Introduction

The objective of this report is to implement a fast and safe rocket landing while respecting actuator limits and state constraints. We use model predictive control to balance performance, accuracy, and stability. Simulation results show that while nominal MPC is able to meet performance requirements, robustness is essential to mitigate uncertainty, which strongly affects feasibility and steady-state performance. We also apply nonlinear MPC on the full dynamics to compare it with linear MPC and evaluate different performance trade-offs.

2 Deliverable 2.1: Separation of the system into independent subsystems

We are able to split the rocket into approximately independent subsystems because each actuator mainly affects a different physical motion. The two gimbal angles tilt the thrust direction. Each tilt is responsible for creating a sideways force in the x-direction and y-direction, respectively. So, they mainly control lateral motion and their corresponding pitch and roll dynamics.

The average thrust P_{avg} mostly changes the magnitude of the thrust along the z vertical direction, controlling the altitude and vertical speed. Whereas, the differential thrust P_{diff} mostly creates a torque imbalance between the propellers which affects the yaw/roll angle γ .

Near the upright hovering condition, which is the equilibrium point about which the system is trimmed, the couplings between these subsystems are small for small angles, meaning the magnitude of the effect of an input on a state from another subsystem is negligible. This justifies analyzing and designing four separate controllers. In order to make this separation valid, we also impose state constraints on the rocket so that it remains within the region of operation where the coupling can be neglected.

We can also look at the state transition and input matrix for the linearized system to infer which inputs affect which systems directly (through the input) or indirectly (through the system dynamics).

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 9.806 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -9.806 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix} \quad B = \begin{bmatrix} -65.47484472 & 0 & 0 & 0 \\ 0 & -65.47484472 & 0 & 0 \\ 0 & 0 & 0 & 0.104 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 9.806 & 0 & 0 \\ -9.806 & 0 & 0 & 0 \\ 0 & 0 & 0.14709 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Figure 1: State transition and input matrix for the linearized system

As seen from the input matrix B for the linearized system about the equilibrium point, the input δ_1 only affects ω_x, v_y directly, and thus α and y indirectly through the state matrix A . The input δ_2 only affects ω_y, v_x directly, and thus β and x indirectly through the state matrix A . The input P_{avg} affects v_z directly and thus z indirectly through the state matrix A . The input P_{diff} affects ω_z directly and thus γ indirectly through the state matrix A . The matrix A is very sparse, and the non-zero entries represent the effect that the states have on each other in a small neighborhood of the equilibrium point, since we are analyzing the linearized system. These entries show that the states can be grouped into 4 subsystems - $\{\omega_y, \beta, v_x, x\}$, $\{\omega_x, \alpha, v_y, y\}$, $\{v_z, z\}$ and $\{\omega_z, \gamma\}$, while the zero blocks from the matrix A can be discarded. The decomposed subsystems, therefore, use only the corresponding rows and columns of A of the states within that subsystem for the separated controller design.

3 Deliverable 3

3.1 MPC Velocity Controllers

3.1.1 Design procedure that ensures recursive constraint satisfaction

For this deliverable, a Δ -formulation around the trim point (x_s, u_s) is adopted (more on this in 3.2), where the MPC problem is expressed in deviation variables $\Delta x = x - x_s$ and $\Delta u = u - u_s$, allowing the constraints to be enforced relative to the equilibrium. Recursive constraint satisfaction is ensured by imposing a terminal constraint $\Delta x_N \in X_f$, where X_f is a positively invariant set for the closed-loop dynamics $\Delta x^+ = (A + BK)\Delta x$ under the stabilizing feedback law $\Delta u = K\Delta x$. The terminal set is constructed to guarantee satisfaction of the original constraints in absolute coordinates, namely $x_s + X_f \subseteq X$ and $KX_f + u_s \subseteq U$. Consequently, after applying the first optimal input, the remaining control sequence can be shifted and completed using the terminal control law, which guarantees the feasibility of the MPC problem at the subsequent time step.

3.1.2 Tuning

The tuning of the cost function was performed iteratively based on closed-loop behavior and constraint satisfaction. Initially, identical weights were assigned to all state components in Q , which led to small, numerically caused, but undesirable violations of the state constraints. It was observed that the attitude angles α and β tended to approach their constraint limits, while angular velocities exhibited large excursions due to the absence of explicit constraints. Consequently, higher weights were assigned to attitude angles and even higher to angular velocities to prevent boundary activation and suppress aggressive rotational dynamics.

The input weight R was chosen to be relatively small in order to avoid excessive penalization of the control effort and to allow sufficiently fast corrective actions, while still ensuring a non-negligible penalty on the input to prevent overly aggressive control behavior. A consistent tuning strategy was applied across all controllers by assigning identical or comparable weights to states representing the same physical quantities. Specifically, the x and y velocity controllers with states $[\omega, \beta, v]$ use $Q = \text{diag}(50, 35, 10)$, the z velocity controller uses $Q = \text{diag}(10)$, and the roll controller with states $[\omega_z, \gamma]$ uses $Q = \text{diag}(50, 10)$. In all cases, the input weight was fixed to $R = \text{diag}(1)$.

The terminal cost Q_f is obtained from the LQR design. The terminal set X_f is chosen as the maximal positively invariant set for the closed-loop system under the LQR feedback, ensuring recursive feasibility by enforcing $\Delta x_N \in X_f$.

3.1.3 Terminal invariant set plots

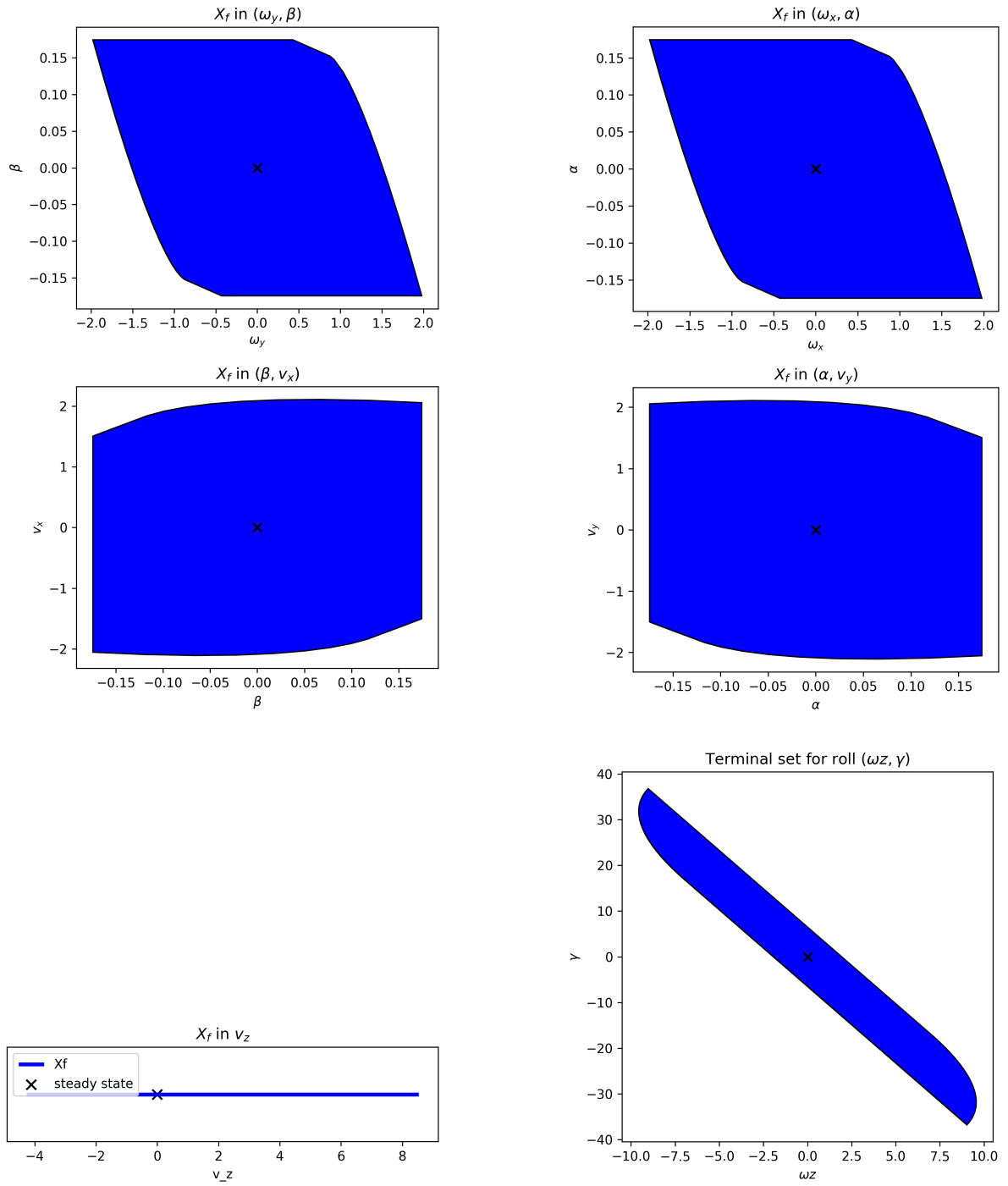


Figure 2: Terminal invariant sets X_f used for recursive feasibility

3.1.4 Open-loop and closed-loop results

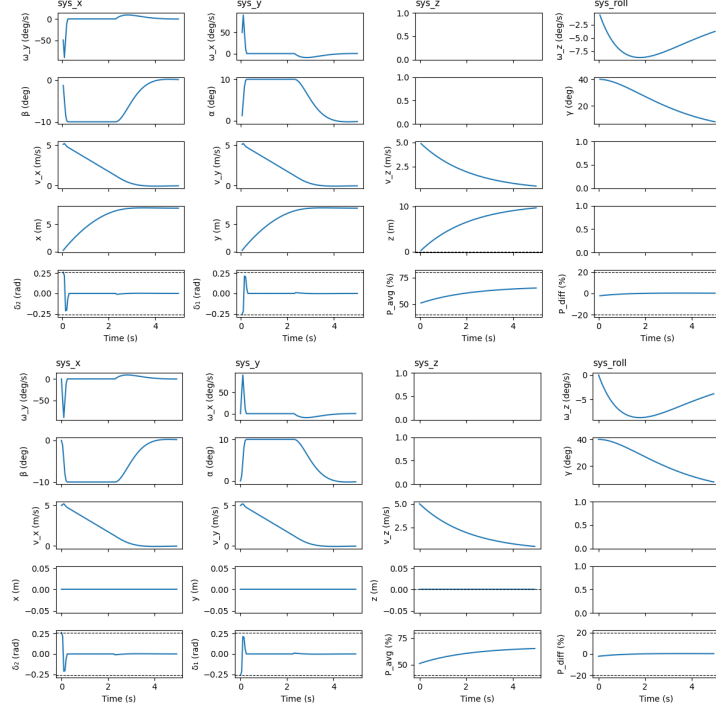


Figure 3: Open-loop and closed-loop responses of the controller

In closed loop (Fig. 3), all controlled velocities converge to zero within the prescribed settling time while respecting the input constraints. The position states x, y, z are not directly controlled, but they still converge. In the open-loop system, the same behavior is also observed. However, the position states x, y, z cannot be assessed, which is consistent with the open-loop setup.

3.2 Tracking

3.2.1 Design procedure and tuning parameters

Like in deliverable 3.1, the delta formulation around the trim point (x_s, u_s) is used:

$$\Delta x_k := x_k - x_s, \quad \Delta u_k := u_k - u_s,$$

with linear dynamics in deviation coordinates

$$\Delta x_{k+1} = A \Delta x_k + B \Delta u_k.$$

Given a desired target state x_{target} , the reference in deviation form is

$$\Delta x_{\text{ref}} := x_{\text{tar}} - x_s,$$

(and $\Delta x_{\text{ref}} = 0$ corresponds to regulation to the trim point). At each time step, we solve the finite-horizon MPC problem

$$\begin{aligned} \min_{\{\Delta x_k, \Delta u_k\}_{k=0}^{N-1}} \quad & \sum_{k=0}^{N-1} [(\Delta x_k - \Delta x_{\text{ref}})^\top Q (\Delta x_k - \Delta x_{\text{ref}}) + \Delta u_k^\top R \Delta u_k] + (\Delta x_N - \Delta x_{\text{ref}})^\top Q_f (\Delta x_N - \Delta x_{\text{ref}}) \\ \text{s.t.} \quad & \Delta x_0 = x_0 - x_s, \\ & \Delta x_{k+1} = A \Delta x_k + B \Delta u_k, \quad k = 0, \dots, N-1, \\ & x_s + \Delta x_k \in X, \quad u_s + \Delta u_k \in U, \quad k = 0, \dots, N-1, \\ & \Delta x_N \in X_f. \end{aligned}$$

In our implementation, the constraints (depending on a system) are enforced explicitly in absolute variables, e.g.

$$\begin{aligned} |\beta_s + \Delta\beta_k| &\leq \beta_{\max}, & |\alpha_s + \Delta\alpha_k| &\leq \alpha_{\max}, \\ |\delta_{2,s} + \Delta\delta_{2,k}| &\leq \delta_{2,\max}, & |\delta_{1,s} + \Delta\delta_{1,k}| &\leq \delta_{1,\max}, \\ |P_{\text{avg},s} + \Delta P_{\text{avg},k}| &\leq P_{\text{avg},\max}, & |P_{\text{diff},s} + \Delta P_{\text{diff},k}| &\leq P_{\text{diff},\max}. \end{aligned}$$

After solving, we apply the first optimal input in absolute coordinates:

$$u_k = u_s + \Delta u_0$$

and repeat at the next time step (receding horizon).

Matrices were left unchanged, as there was no need for additional tuning.

3.2.2 Open-loop and closed-loop results

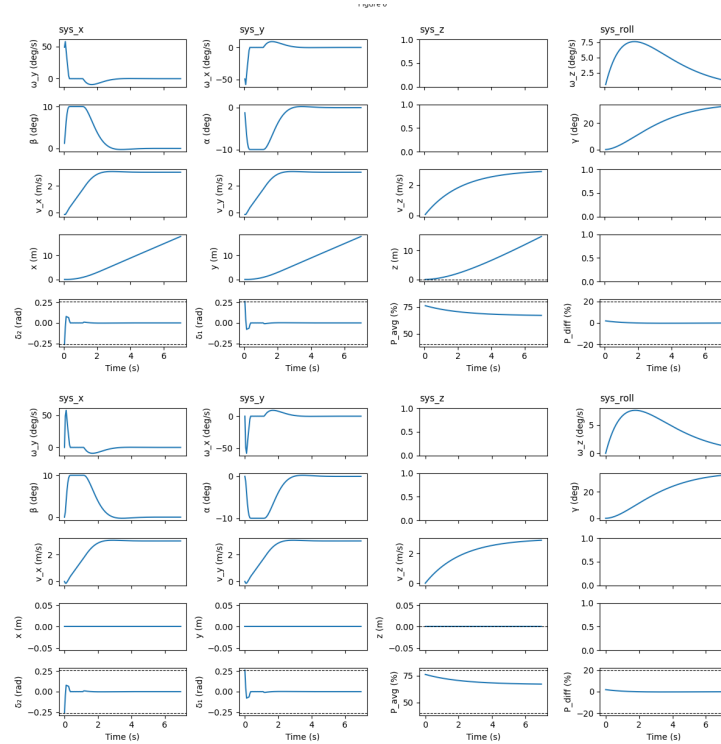


Figure 4: Open-loop and closed-loop responses of the controller

In Fig. 4, successful tracking of the desired velocities and roll angle to 35° is observed in both open and closed loop. The position states x, y, z , which can be seen in the closed-loop plot, continue to increase because converged velocities are non-zero positive. The velocities in x, y, z all converge to 3m/s.

3.3 PID contoler

3.3.1 Outer-loop PI Position Controller

In order to achieve position tracking, we extend the velocity-tracking MPC framework developed in Deliverable 3.2 with an outer-loop position controller implemented using the provided `PIControl` class. The resulting control scheme follows a cascaded architecture.

The outer loop consists of three independent PI controllers, one for each Cartesian coordinate x, y , and z . Each PI controller computes a velocity reference based on the position error

$$e_p(t) = p^{ref} - p(t),$$

according to the control law

$$v^{ref}(t) = K_p e_p(t) + K_i \int_0^t e_p(\tau) d\tau,$$

with fixed gains and sampling time $T_s = 0.05$ s. The output is saturated to predefined bounds to ensure feasible velocity commands.

The resulting velocity reference vector

$$v^{ref} = [v_x^{ref} \quad v_y^{ref} \quad v_z^{ref}]^T$$

is passed directly to the inner-loop MPC velocity controllers designed in Deliverable 3.2, which track these setpoints while handling system dynamics and input constraints.

No additional tuning or structural modifications are required, as the PI controller is provided in the project template and is fully compatible with the existing MPC interface.

3.3.2 Closed-loop results

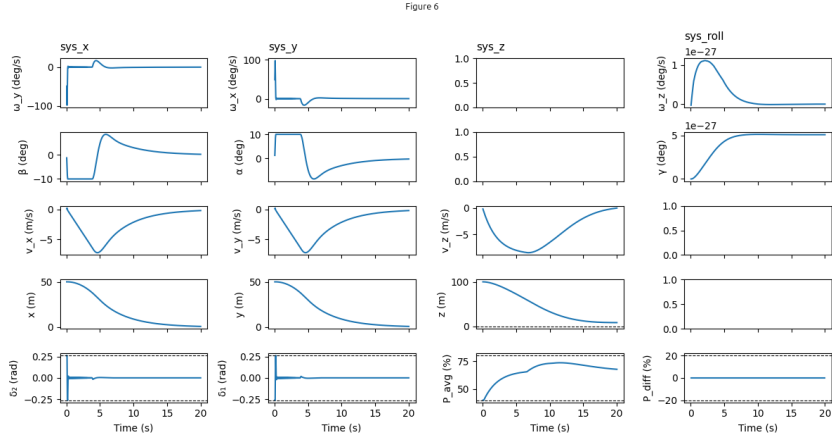


Figure 5: Closed-loop responses of the controller

Fig. 5 shows the closed-loop response with the position controller, where the desired position and roll angle are successfully tracked. All states converge smoothly from (50,50,100) to (0,0,10) and roll=0°, resulting in a stable closed-loop behavior.

4 Deliverable 4.1: Simulation with Nonlinear Rocket

4.1 Design and tuning

In the simulation of the linear system, we assume that the 4 subsystems of the rocket are independent and decoupled with linear dynamics. For our assumptions to be valid, we constrain our states to a region where the dynamics can be approximated as linear. For the nonlinear case, the subsystems with coupled dynamics affect each other, and the subsystems can no longer be considered completely independent. The simulation also lets go of the linearity assumptions, and therefore, the design we use for the linear system may no longer work when simulated for the nonlinear rocket, especially far away from the trip position.

As a consequence, constraint violations were observed in the attitude states α and β , causing the rocket to exceed the 10° limit below which the modeling assumptions are valid. As a result, the rocket toppled and entered an accelerated downward trajectory, rendering the problem infeasible. The aggressive controller induced large violations in the attitude states α and β , which ultimately caused the rocket to flip.

To ensure completion of the maneuver, the penalties on the angular velocities ω_x and ω_y are increased, and slack variables are introduced in the x and y subsystem constraints. The weights on the states and inputs used in the cost function, as well as the modified constraints incorporating the slack variables,

are given below 4.1.

A combination of linear and quadratic costs is applied to the slack variables introduced in the constraints on α and β . The quadratic weight was first increased to reduce constraint violations until further increases had no effect, indicating that the slack variables were already small and that the quadratic penalty $\epsilon^\top S \epsilon$ had become negligible. The linear weight S_{lin} was then increased to further suppress the remaining violations, since the linear term continues to penalize even very small nonzero slacks. This approach pushes the solution closer to hard-constraint behavior without excessively increasing quadratic weights or degrading numerical conditioning.

Subsystems X and Y The lateral subsystems use identical weighting matrices and slack penalties. For both subsystems,

$$Q = \begin{bmatrix} 2000 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 10 \end{bmatrix}, \quad R = 1, \quad S = 500, \quad S_{\text{lin}} = 1000.$$

Soft constraints are imposed on the attitude states,

$$|\beta_s + \Delta\beta| \leq 10^\circ + \varepsilon \quad (\text{Subsystem X}), \quad |\alpha_s + \Delta\alpha| \leq 10^\circ + \varepsilon \quad (\text{Subsystem Y}), \quad \varepsilon \geq 0.$$

The stage cost is given by

$$J = \sum_{i=0}^{N-1} (\Delta x_i^\top Q \Delta x_i + \Delta u_i^\top R \Delta u_i + \varepsilon^\top S \varepsilon + S_{\text{lin}} \varepsilon) + \Delta x_N^\top Q_f \Delta x_N.$$

Subsystem Roll. The roll subsystem is weighted using

$$Q = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}, \quad R = 1.$$

Subsystem Z. For the vertical subsystem,

$$Q = 100, \quad R = 1.$$

4.2 Plots

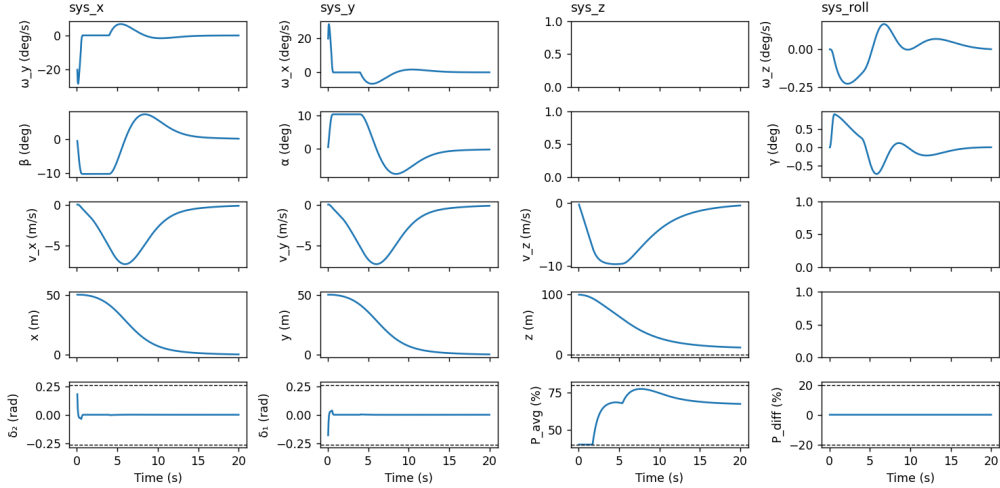


Figure 6: Closed loop simulation with nonlinear rocket

After adding the slack variables and tuning the cost matrices for them, the constraint violation in α and β is limited to around 1° . The positions x, y, z converge to the given targets (0,0,10)m. The roll angle γ converges to 0° . The angles α and β are allowed to momentarily cross their 10° limit to allow the problem to be feasible under model mismatch.

5 Deliverable 5: Offset-free tracking

5.1 Offset-free tracking in z via constant disturbance

Motivation for offset-free tracking. When simulating the controller from Part 4 with a mismatched rocket mass, a steady-state offset is observed in the vertical velocity v_z . In particular, Fig. 7 shows a residual offset of approximately 1.73 m/s, indicating that the controller is unable to reject the steady-state bias induced by the modeling error.

This behavior is expected, as the controller in Part 4 is designed using the nominal model and does not account for constant disturbances entering the vertical dynamics. As a result, the controller cannot compensate for the mass mismatch, leading to steady-state tracking errors.

To address this issue, the z -subsystem is modeled as being affected by a constant but unknown disturbance, capturing the effect of the mass mismatch. The discrete-time dynamics are therefore written as

$$x^+ = Ax + Bu + B_d d,$$

where d is constant and unknown.

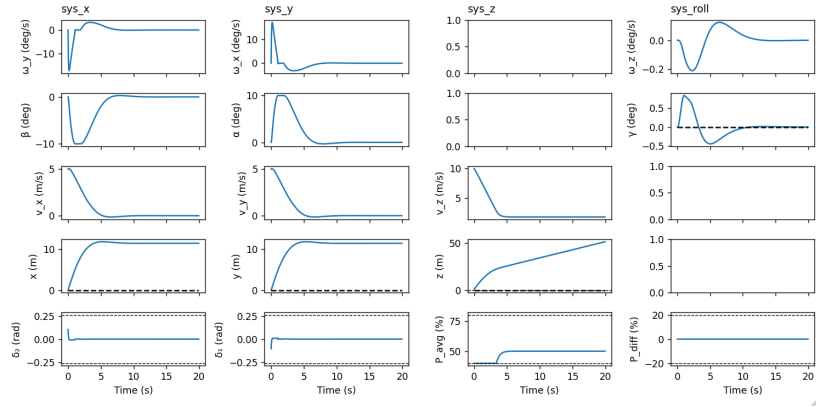


Figure 7: Closed-loop response using the controller from Part 4, exhibiting a steady-state offset.

Why the disturbance term remains in deviation coordinates. Although the controller is formulated in deviation variables with respect to the nominal trim point (x_s, u_s) , the disturbance term does not vanish from the dynamics. This is because the trim point is computed using the nominal model

$$x_s = Ax_s + Bu_s,$$

which does not account for the unknown disturbance. When a constant disturbance d is present in the true system dynamics,

$$x_{k+1} = Ax_k + Bu_k + B_d d,$$

substituting $x_k = x_s + \Delta x_k$ and $u_k = u_s + \Delta u_k$ yields

$$\Delta x_{k+1} = A\Delta x_k + B\Delta u_k + B_d d.$$

Therefore, the delta formulation only removes steady-state components that are explicitly captured by the nominal trim point. As the disturbance is unknown, it must be retained and estimated online, and its estimate is injected into the MPC prediction model to generate the required steady-state input correction and achieve offset-free tracking.

Augmented observer executed online in `get_u`. To estimate the unknown constant disturbance, an augmented state is introduced:

$$z_k := \begin{bmatrix} \Delta x_k \\ d_k \end{bmatrix}, \quad d_{k+1} = d_k.$$

The augmented model is therefore

$$z_{k+1} = \underbrace{\begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix}}_{A_{\text{hat}}} z_k + \underbrace{\begin{bmatrix} B \\ 0 \end{bmatrix}}_{B_{\text{hat}}} \Delta u_k, \quad y_k = \underbrace{\begin{bmatrix} I & 0 \end{bmatrix}}_{C_{\text{hat}}} z_k.$$

A discrete-time Luenberger observer is applied at every call of `get_u`:

$$\hat{z}_{k+1} = A_{\text{hat}} \hat{z}_k + B_{\text{hat}} \Delta u_{k-1} + L(y_k - C_{\text{hat}} \hat{z}_k).$$

The observer gain L is computed via pole placement on the augmented system. The poles are selected as 0.7 and 0.6, yielding a stable estimator with sufficiently fast convergence while avoiding excessive sensitivity to noise. After the update, the estimates are extracted as

$$\hat{z}_{k+1} = \begin{bmatrix} \widehat{\Delta x}_{k+1} \\ \hat{d}_{k+1} \end{bmatrix},$$

and both $\widehat{\Delta x}$ and $\hat{d}$ are then used by the MPC optimization.

MPC prediction model using the estimated disturbance. At each time step, the MPC problem is formulated in deviation variables with decision variables $(\Delta x_k, \Delta u_k)$ and is initialized with the estimated deviation state:

$$\Delta x_0 = \widehat{\Delta x}_k.$$

A constant disturbance estimate $\hat{d}_k$ is assumed over the prediction horizon and injected into the dynamics:

$$\Delta x_{k+1} = A \Delta x_k + B \Delta u_k + B_d \hat{d}_k.$$

The quadratic objective is

$$\min_{\{\Delta x_k, \Delta u_k\}} \sum_{k=0}^{N-1} \left((\Delta x_k - \Delta x^{\text{ref}})^\top Q (\Delta x_k - \Delta x^{\text{ref}}) + \Delta u_k^\top R \Delta u_k \right) + (\Delta x_N - \Delta x^{\text{ref}})^\top Q_f (\Delta x_N - \Delta x^{\text{ref}}).$$

and the thrust constraint is enforced in absolute form:

$$P_{\min} \leq P_s + \Delta u_k \leq P_{\max}.$$

Finally, the applied input is recovered as $u_k = u_s + \Delta u_k$.

Comparison with controller from part 4. As mentioned, when comparing the controllers from Part 4 and Part 5, a closer inspection reveals a steady-state offset of a 1.73 m/s in v_z for the controller in Part 4, as can be seen in Fig. 7. This behavior is expected, as the controller in Part 4 does not account for the constant disturbance.

Tuning choices. The same weights for Q, R, Q_f, S, S_{lin} are used as in Part 4. The terminal set is also dropped, as stated in the deliverable. The estimator speed is mainly determined by the chosen observer poles (0.7, 0.6): faster poles would adapt $\hat{d}$ more quickly but may amplify noise, while slower poles would delay disturbance rejection and thus delay offset removal. This can also be concluded from Fig. 8.

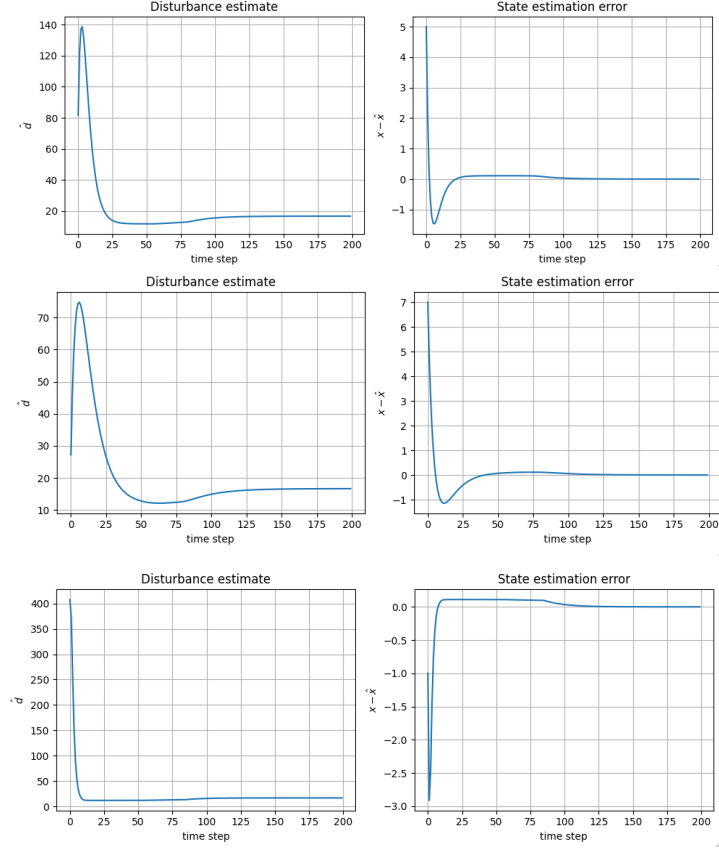


Figure 8: Disturbance estimates $\hat{d}$ and state estimation error $\tilde{x} = x - \hat{x}$ for different observer pole placements.

The top pair of plots corresponds to observer poles at $(0.6, 0.7)$, showing a well-balanced convergence of both the disturbance estimate and the state estimation error. The middle pair corresponds to slower poles $(0.4, 0.6)$, yielding smoother responses at the cost of significantly slower convergence. The bottom pair shows faster observer poles $(0.8, 0.9)$, resulting in quicker convergence but with a big initial error.

With a zero fuel rate, the rocket mass is constant, and the mass mismatch manifests as a constant disturbance in the vertical dynamics. As shown in Fig. 8, the estimate $\hat{d}$ converges to a constant value, and V_z converges to zero, demonstrating correct identification of the steady-state bias by the observer. Thus, for a constant rocket mass, the disturbance is time-invariant and can be accurately captured with this model.

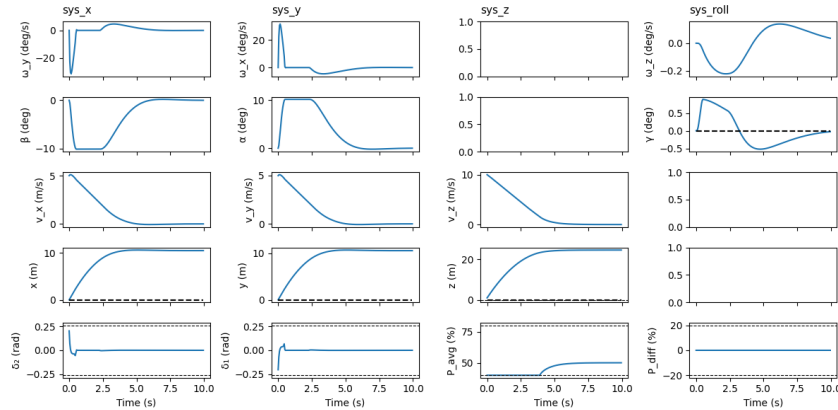


Figure 9: Closed-loop offset-free tracking in the presence of a constant disturbance, showing convergence to the reference without steady-state error.

5.2 Deliverable 5.2: Offset-free tracking with changing mass of the rocket

Due to the non-zero fuel rate, the rocket mass varies over time, which invalidates the assumption of a constant disturbance in the vertical dynamics. As shown in Fig. 10, the disturbance estimate exhibits a slow drift, indicating a time-varying bias caused by mass variation.

A more appropriate formulation is to model the disturbance as a slowly time-varying process, for instance, using a random-walk model $d_{k+1} = d_k + w_k$, which enables the estimator to track this bias and improves offset-free tracking of the vertical velocity. The disturbance estimate exhibits an approximately linear drift over time, which further supports the use of this modeling approach.

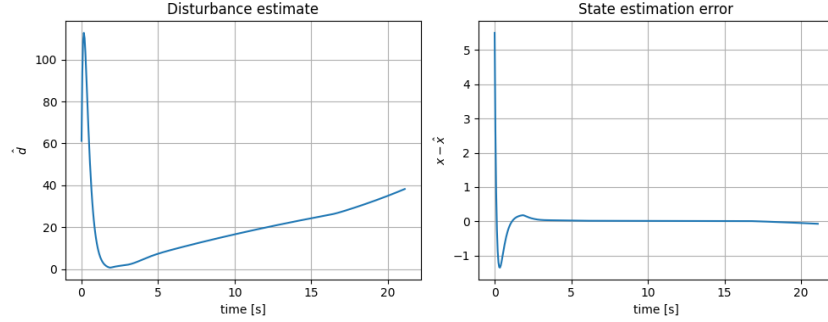


Figure 10: Disturbance estimates $\hat{d}$ and state estimation error $\tilde{x} = x - \hat{x}$ for pole placement (0.7,0.85)

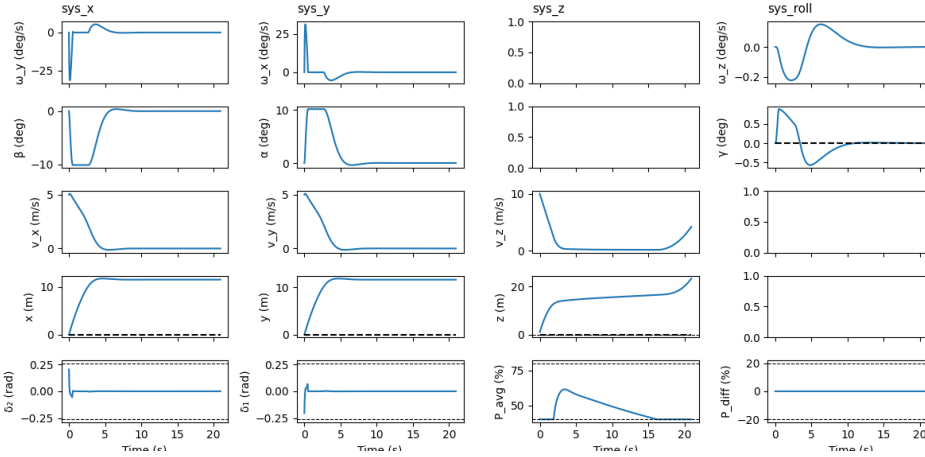


Figure 11: Closed-loop offset-free tracking in the presence of a varying disturbance

Time-varying mass behavior. At the beginning of the simulation, the rocket has a large positive vertical velocity while the reference velocity is zero. MPC initially aims to reduce the upward velocity as quickly as possible. This is achieved by minimizing the thrust, and due to the input constraint $P_{\text{avg}} \geq 40\%$, the optimal control input is saturated at its lower bound.

The thrust force F_z is smaller than the gravitational force mg , resulting in a negative vertical acceleration. This behavior is intentional and consistent with the velocity-tracking objective.

During this phase, modifying the observer dynamics does not affect the control action, as the optimal input is already determined by the saturation constraint. Using faster observer poles mainly introduces larger transient errors at the beginning, without providing additional performance benefits.

As the vertical velocity decreases, the controller increases P_{avg} to balance the decreasing gravitational force due to the reduced mass.

In the middle of the simulation, P_{avg} increases to approximately 60%, bringing the thrust force close to the gravitational force. As a result, the vertical acceleration approaches zero, the system stabilizes, entering a hover phase.

Towards the end of the simulation, the mass continues to decrease due to fuel consumption. However, the minimum throttle constraint $P_{\text{avg}} \geq 40\%$ prevents the controller from further reducing the thrust. As a result, the thrust force F_z becomes larger than the gravitational force mg , which gets smaller and smaller due to the smaller mass. This imbalance produces a positive vertical acceleration, leading to an increase in a_z , a growing vertical velocity v_z , and an unintended upward motion.

This behavior is caused by the combination of input saturation and the decreasing mass, which prevents the controller from maintaining zero vertical acceleration.

6 Deliverable 6: Robust MPC for Landing

6.1 Robust tube MPC for subsystem Z

6.1.1 Design and tuning

For the disturbance set

$$W = [-15, 5]$$

, since the disturbance affects the subsystem through the input channel, the corresponding polyhedron W was multiplied by the subsystem's discretized input matrix B_d . For a stabilizing input gain matrix K , the closed-loop state transition matrix

$$A_{cl} = A_d + B_d K$$

is used to compute the minimal robust invariant set E . We chose the infinite horizon LQR matrix K and the corresponding infinite horizon cost matrix Q_f . In order to ensure recursive feasibility of the problem, the terminal set was chosen as the maximal positively invariant set for the control law

$$u = Kx.$$

We choose a positive definite terminal cost matrix Q_f such that the terminal cost

$$V_f(\Delta x) = \Delta x^\top Q_f \Delta x$$

satisfies

$$\begin{aligned} V_f(\Delta x^+) - V_f(\Delta x) &\leq -J(\Delta x, \kappa_f(\Delta x)), \\ \Delta x^\top (A_{cl}^\top Q_f A_{cl} - Q_f) \Delta x &\leq -\Delta x^\top (Q + K^\top R K) \Delta x. \end{aligned}$$

Accordingly, Q_f was chosen as the solution of the discrete Lyapunov equation

$$A_{cl}^\top Q_f A_{cl} - Q_f + (Q + K^\top R K) = 0,$$

where Q and R are the state and input quadratic cost matrices.

The state and input constraints were tightened using the set E as

$$X \ominus E \quad \text{and} \quad U \ominus KE,$$

respectively.

6.1.2 MPC formulation

Variables:

Nominal trajectory: $\bar{x}_0, \bar{x}_1, \dots, \bar{x}_N$

Inputs: $v_0, v_1, \dots, v_{N-1}$

Constraints:

$$\begin{aligned}\bar{x}_{i+1} &= A\bar{x}_i + Bv_i, \\ \bar{x}_i &\in X \ominus E, \quad i = 0, 1, \dots, N, \\ v_i &\in U \ominus KE, \quad i = 0, 1, \dots, N-1, \\ \Delta x_0 - \bar{x}_0 &\in E, \\ \bar{x}_N &\in \chi_f.\end{aligned}$$

Cost function:

$$J = \sum_{i=0}^{N-1} (\bar{x}_i^\top Q \bar{x}_i + v_i^\top R v_i) + \bar{x}_N^\top Q_f \bar{x}_N,$$

with

$$Q = \begin{bmatrix} 100 & 0 \\ 0 & 100 \end{bmatrix}, \quad R = [0.5].$$

The values of Q and R were tuned using trial and error while observing the performance of the controller and the settling time in the extreme and random disturbance cases. The weight on the vertical velocity was increased to avoid rapid descents during the landing maneuver.

Control input:

$$u = u_s + K(\Delta x_0 - \bar{x}_0) + v_0,$$

where u_s is the input at the point around which the system has been linearized. This term must be added since the tube MPC is formulated in Δ coordinates using the linearized system.

6.1.3 Minimal invariant set E , terminal set χ_f and tightened input set $\tilde{U}$

All the sets below are in global coordinates and not in the Δ coordinates. The equilibrium state was added to the sets (if the set was given by the inequalities $Hx \leq h$ in the Δ formulation, the sets plotted here are given by the inequalities $Hx \leq h + Hx_s$)

The tightened input set is

$$\tilde{U} : P_{\text{avg}} \in [45.8185985, 62.90350418].$$

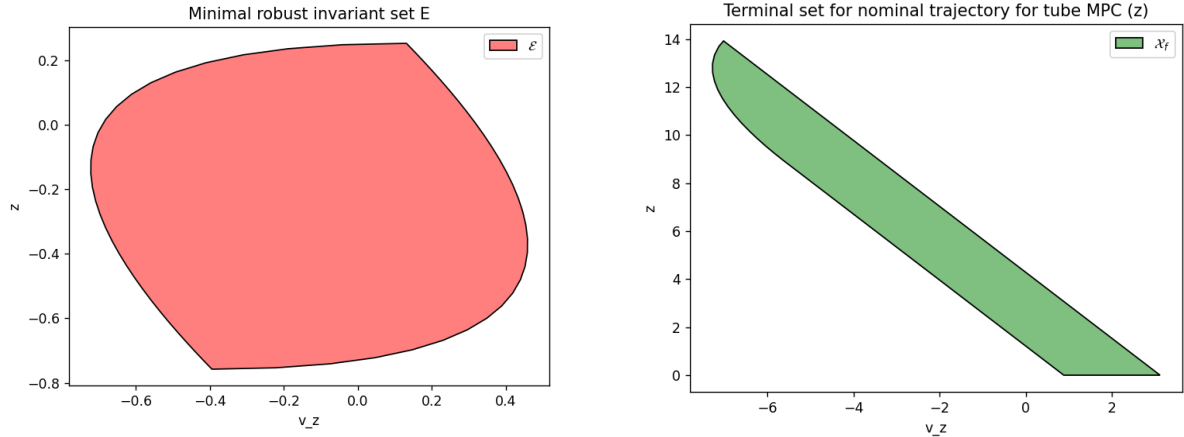


Figure 12: Invariant set E (left) and terminal set χ_f (right).

6.1.4 Plots

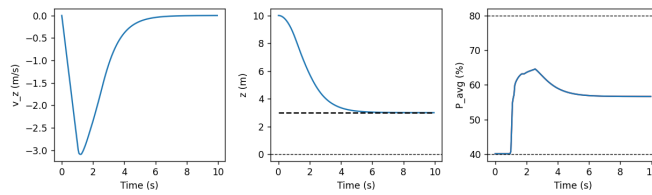


Figure 13: Closed loop plot for no noise case

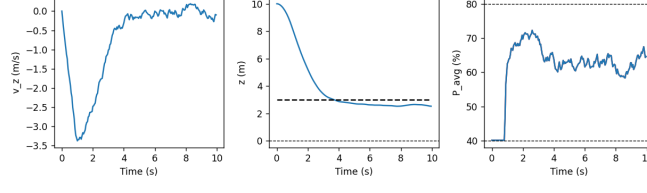


Figure 14: Closed loop plot for random disturbances

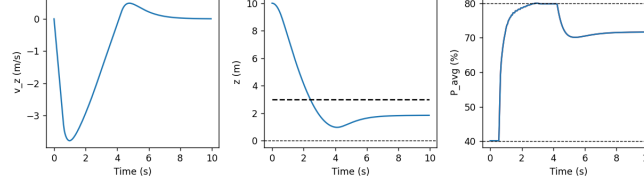


Figure 15: Closed loop plot for extreme disturbances

In the random disturbance case, random noise persistently pushes the actual system state away from the nominal trajectory computed by the MPC. The feedback controller then acts to steer the state back toward the center of the tube. Because the disturbance is time-varying and does not decay, this correction process never fully settles, leading to low-amplitude bounded oscillations around the reference.

In the extreme disturbance case, there is a constant offset in the value that the state converges to and the desired setpoint. This could be due to the fact that in the extreme noise case, the disturbance is a constant disturbance at $w = -15$. At the state to which the rocket converges (let this state be called z_0), given the chosen costs, the nominal trajectory and designed input is such that the effect of $K(\Delta x_0 - \bar{x}_0)$ and v_0 compensate for each other in such a way that the computed input $u = x_s + K((z_0 - x_s) - \bar{x}_0) + v_0$ becomes an equilibrium input for the system $x^+ = Ax + Bu + B\bar{w}$, where $\bar{w} = w_{min}$. Since the nominal trajectory and input planned are optimal and deterministic, for a given initial condition, the $\bar{x}_0$ and v_0 designed would be the same. This would contribute towards regulation if the noise were unpredictable (as seen in the random noise case), but here, since the disturbance is constant, the state eventually converges to a new equilibrium, which is a function of the MPC parameters and the constant disturbance. This behavior is expected in tube MPC, since the controller is designed to guarantee constraint satisfaction and boundedness for all admissible disturbances, rather than asymptotic convergence of the actual state to the nominal reference.

6.2 Deliverable 6.2: Robust Tube MPC for landing with nominal position controllers for X, Y, roll

The structure of the merged controller is the same as the velocity controllers used in Section 3 of the project, with the X and Y subsystems having one additional dimension due to the inclusion of position states. The terminal cost is the infinite-horizon LQR cost.

The cost is quadratic in $(\Delta x - \Delta x_{ref})$, where Δx_{ref} denotes the desired target state at the end of the maneuver. The weights were selected by trial and error, with the largest penalties assigned to the angular velocity states in order to avoid overly aggressive maneuvers. Soft constraints are enforced using a combination of quadratic and linear penalties on the slack variables, $\epsilon^T S \epsilon + S_{lin} \epsilon$. The parameter values are identical to those used in Section 4. Since the controllers are applied to the nonlinear model, the terminal constraints for the nominal controllers are omitted.

The state and input cost matrices for the different subsystems are given below:

Subsystem X

$$Q = \begin{bmatrix} 200 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix}, \quad R = [0.05], \quad S = 500, \quad S_{lin} = 1000$$

Subsystem Y

$$Q = \begin{bmatrix} 70 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 \\ 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 10 \end{bmatrix}, \quad R = [0.5], \quad S = 500, \quad S_{lin} = 1000$$

Subsystem roll

$$Q = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}, \quad R = [0.1].$$

6.2.1 Plots

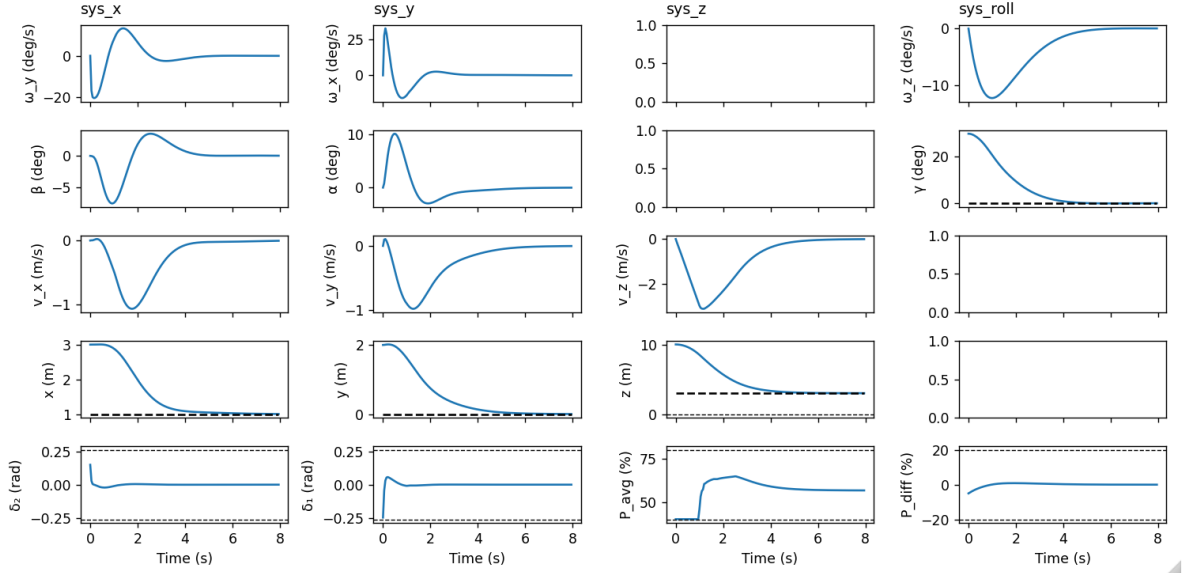


Figure 16: Closed loop plot for merged MPC controller

The positions x, y, z and the roll angle γ converge to the desired target $(1, 0, 3, 0^\circ)$ from $(3, 2, 10, 30^\circ)$ in 4 seconds.

7 Deliverable 7: Nonlinear MPC

7.1 Design Procedure

In this section, we design a nonlinear model predictive controller (NMPC) to regulate the rocket to a landing equilibrium while respecting constraints. The continuous nonlinear model has a 12 dimensional state vector and a 4-dimensional input vector:

$$\mathbf{x} = [\omega_x \ \omega_y \ \omega_z \ \alpha \ \beta \ \gamma \ v_x \ v_y \ v_z \ x \ y \ z]^\top \in \mathbb{R}^{12}. \quad (1)$$

$$\mathbf{u} = [\delta_1 \ \delta_2 \ P_{avg} \ P_{diff}]^\top \in \mathbb{R}^4. \quad (2)$$

The aim of the NMPC controller is to regulate the rocket around a landing equilibrium point $(\mathbf{x}_s, \mathbf{u}_s)$ corresponding to zero velocities and upright attitude at

$$\mathbf{p}_s = [1 \quad 0 \quad 3]^\top. \quad (3)$$

With these conditions, the trim computation finds $(\mathbf{x}_s, \mathbf{u}_s)$, and the NMPC cost then penalizes any deviation from this equilibrium.

(1) Discrete-time Prediction Model (Runge–Kutta 4). We first discretize the continuous model using a 4th order Runge-Kutta integrator with sampling time T_s :

$$\mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k), \quad (4)$$

where

$$\mathbf{k}_1 = f(\mathbf{x}_k, \mathbf{u}_k), \quad (5)$$

$$\mathbf{k}_2 = f\left(\mathbf{x}_k + \frac{T_s}{2}\mathbf{k}_1, \mathbf{u}_k\right), \quad (6)$$

$$\mathbf{k}_3 = f\left(\mathbf{x}_k + \frac{T_s}{2}\mathbf{k}_2, \mathbf{u}_k\right), \quad (7)$$

$$\mathbf{k}_4 = f(\mathbf{x}_k + T_s\mathbf{k}_3, \mathbf{u}_k), \quad (8)$$

$$F(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{x}_k + \frac{T_s}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (9)$$

(2) Finite-Horizon NMPC Formulation At each time step t_0 , given the measured state $\mathbf{x}_0 = \hat{\mathbf{x}}(t_0)$, we solve the nonlinear program

$$\min_{\{\mathbf{x}_k\}_{k=0}^N, \{\mathbf{u}_k\}_{k=0}^{N-1}} \sum_{k=0}^{N-1} \left[(\mathbf{x}_k - \mathbf{x}_s)^\top Q (\mathbf{x}_k - \mathbf{x}_s) + (\mathbf{u}_k - \mathbf{u}_s)^\top R (\mathbf{u}_k - \mathbf{u}_s) \right] + (\mathbf{x}_N - \mathbf{x}_s)^\top P (\mathbf{x}_N - \mathbf{x}_s) \quad (10)$$

$$\text{s.t. } \mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \quad (11)$$

$$\mathbf{x}_0 = \hat{\mathbf{x}}(t_0). \quad (12)$$

$$z_k \geq 0, \quad \forall k, \quad (13)$$

$$|\beta_k| \leq 80^\circ, \quad \forall k, \quad (14)$$

$$|\delta_{1,k}| \leq 0.26, \quad |\delta_{2,k}| \leq 0.26, \quad \forall k, \quad (15)$$

$$40 \leq P_{\text{avg},k} \leq 80, \quad |P_{\text{diff},k}| \leq 20, \quad \forall k. \quad (16)$$

(3) Terminal cost-design To approximate the infinite horizon cost, we include a terminal cost matrix P . We compute P once from a local linear model following these steps:

1. Linearize and discretize the system at $(\mathbf{x}_s, \mathbf{u}_s)$ to obtain (A_d, B_d) .
2. Solve the discrete-time LQR problem to get a feedback K using (A_d, B_d, Q, R) .
3. Form $A_{\text{cl}} = A_d + B_d K$ and $\bar{Q} = Q + K^\top R K$.
4. Solve the discrete Lyapunov equation to obtain P :

$$A_{\text{cl}}^\top P A_{\text{cl}} - P + \bar{Q} = 0. \quad (17)$$

This terminal penalty encourages stable convergence near the trim point even when the horizon is finite.

(4) Nonlinear Optimizer The problem is implemented in CasADi Opti and solved with IPOPT at every control step. The controller only applies the first optimal input after solving.

7.2 Choice of Tuning Parameters

The primary tuning knobs of the NMPC are the stage cost's matrices Q and R . These two matrices determine how the optimizer trades off tracking performance and effort.

State weighting matrix Q . We use a diagonal state weighting matrix consistent with the state ordering

$$[\omega_x, \omega_y, \omega_z, \alpha, \beta, \gamma, v_x, v_y, v_z, x, y, z].$$

The selected weights are

$$Q = \text{diag}(500, 500, 500, 200, 200, 200, 50, 50, 55, 100, 200, 90). \quad (18)$$

The weights in Q were tuned following a systematic approach. We incrementally increased the weights on $\omega_x, \omega_y, \omega_z$ to strongly penalize fast rotational motion, prevent oscillation, and decrease settling time. Penalties on (α, β, γ) are needed to keep the rocket upright and enforce stable landing. To reduce overshoot and oscillatory behavior, we apply moderate penalties on (v_x, v_y, v_z) without making the trajectory too slow. Finally, to enforce convergence to the desired landing position, we tune the weights accordingly on (x, y, z) .

Input weighting matrix R . The input weighting matrix on $[\delta_1, \delta_2, P_{\text{avg}}, P_{\text{diff}}]$ is chosen as

$$R = \text{diag}(3000, 3000, 0.2, 20). \quad (19)$$

Very large weights on δ_1, δ_2 are applied to discourage rapid and large gimbal deflections. This makes the controller's attitude control smoother and not excite oscillations. A small weight applied on P_{avg} and a higher penalty on P_{diff} is mainly to modulate thrust freely, control altitude but limit aggressive differential. This helps to prevent oscillations.

The same (Q, R) are also applied when computing the terminal penalty P for consistency with the stage cost.

7.3 Performance Analysis

In the open-loop prediction (Fig. 17), the NMPC plans a smooth landing trajectory with a well-damped transient. Roll γ is corrected but not yet fully zero by the end of the horizon. However, in the closed-loop response (Fig. 18), the states x, y converge to their references in about 2.5 s and remain stable afterwards, and the state z in about 3 s. The roll γ settles slower than the position states.

Overall, the closed-loop NMPC achieves a faster and more reliable convergence in position than the open-loop response. The settling time requirement for landing is settled in less than 4 seconds in the closed-loop response.

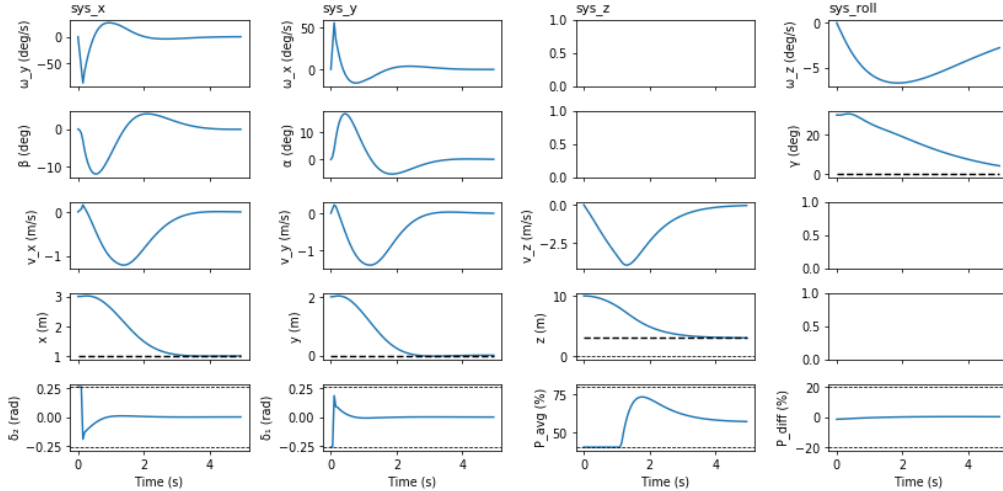


Figure 17: Open loop plot for NMPC

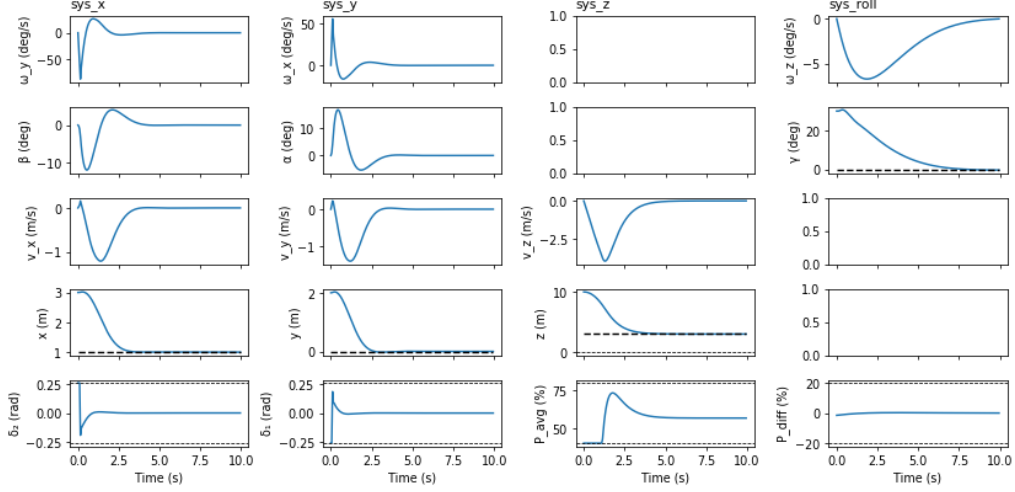


Figure 18: Closed loop plot for NMPC

7.4 NMPC vs Robust MPC

NMPC uses the full nonlinear rocket model to predict and choose the best inputs. This means that it gives a more accurate behavior as it naturally captures the coupling between the attitude and position. Whereas the Robust Tube MPC in Part 6.2 with the three nominal MPCs is based on a linearized model that is split into smaller controllers. Its advantage is that it is faster to solve since it is a QP and is able to handle disturbances more safely by tightening the constraints.

Tube/Nominal MPC's disadvantage is that it can be more conservative, while NMPC can be less predictable and heavier to compute. This is because it solves a nonconvex nonlinear optimization.

Plots Analysis From the plots, both controllers land the rocket with stability. Fig. 16 and Fig. 18 show that x and y reach their targets in about 2-3 s and z reaches about 3-4 s smoothly with no big oscillations. Roll γ decays more slowly when settling than the position states in both cases. Compared to the tube/nominal MPC, the NMPC plot shows a more aggressive initial transient due to nonlinear coupling. On the other hand, the tube/nominal MPC response appears more conservative with smoother and smaller control actions.